

ARITHMETIC EXPRESSION EVALUATOR

User's Manual

Version 1.0

Arithmetic Expression Evaluator	Version: 1.0
User's Manual	Date: 01/05/2026
EECS348-G10-UM	

Revision History

Date	Version	Description	Author
01/05/2026	1.0	Section 1 and 2 plus revision on the template.	Ayushma Koirala
5/05/2026	1.0	Section 3,4,5,6,7,8	Nikki Sotoodi
5/06/2026	1.0	Review and tools added to section 3-8.	Ayushma Koirala

Arithmetic Expression Evaluator	Version: 1.0
User's Manual	Date: 01/05/2026
EECS348-G10-UM	

Table of Contents

1. Purpose	4
2. Introduction	4
3. Getting started	4
4. Advanced features	4
5. Troubleshooting	4
6. Example of uses	4
7. Glossary	5
8. FAQ	5

Arithmetic Expression Evaluator	Version: 1.0
User's Manual	Date: 01/05/2026
EECS348-G10-UM	

USER MANUAL

1. Purpose

The purpose of this User's Manual is to provide instructions for installing, running, and using the Arithmetic Expression Evaluator. This manual explains the features of the software, how to enter arithmetic expressions, how results are displayed, and how errors are handled. It is intended to help users understand and operate the software effectively.

2. Introduction

The Arithmetic Expression Evaluator is a simple C++ program that solves math expressions entered by the user. The program follows normal math rules like PEMDAS and can handle parentheses, decimals, and different arithmetic operators.

The evaluator supports:

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)
- Modulo (%)
- Exponentiation (**)

The program also checks for errors such as invalid input, missing values, or division by zero.

Features of the software include:

- Tokenization of arithmetic expressions
- Parsing expressions according to precedence rules
- Evaluation of integer and floating-point expressions
- Error detection for invalid syntax and runtime errors
- Clear output messages for results and errors

System Requirements

To run the Arithmetic Expression Evaluator, the following are required:

- A computer running Windows, macOS, or Linux
- A C++17 compatible compiler (such as g++)
- A command-line terminal or IDE terminal

Arithmetic Expression Evaluator	Version: 1.0
User's Manual	Date: 01/05/2026
EECS348-G10-UM	

Installation and Running:

1. Compile the source code using a C++17 compiler:

```
g++ main.cpp -o evaluator
```

2. Run the executable:

For Windows:

```
evaluator.exe
```

For macOS/Linux:

```
./evaluator
```

3. Enter a math expression when prompted.

3. Getting Started

The Arithmetic Expression Evaluator is a command-line application that allows users to enter arithmetic expressions and receive evaluated results.

How to Compile the Program

Step 1: Open a terminal in the project directory.

Step 2: Compile the program using the following command:

```
g++ *.cpp -o evaluator
```

Step 3: Run the program:

Arithmetic Expression Evaluator	Version: 1.0
User's Manual	Date: 01/05/2026
EECS348-G10-UM	
./evaluator	

How to Use the Program

After running the program, the user can enter an arithmetic expression directly into the terminal.

Enter an Expression

Type any valid arithmetic expression and press Enter.

Example:

5 + 2

View the Result

The program will calculate and display the answer.

Output:

7

The program supports:

- Whole numbers
- Decimal numbers
- Parentheses
- Positive and negative numbers
- Multiple operators in one expression

Examples:

(3 + 2) * 5

Arithmetic Expression Evaluator	Version: 1.0
User's Manual	Date: 01/05/2026
EECS348-G10-UM	

```
-4 + 7
1.5 + 2.5
2 ** 3
```

4. Advanced features

4.1 Operator Precedence

The evaluator follows standard PEMDAS rules:

1. Parentheses
2. Exponentiation
3. Multiplication, Division, and Modulo
4. Addition and Subtraction

4.2 Unary Operators

The software supports unary plus and unary minus operators.

Example:

Input: $-5 + 10$

Output: 5

4.3 Nested Parentheses

Users may enter expressions with multiple levels of parentheses.

Example:

Input: $((2 + 3) * (4 - 1))$

Output: 15

Arithmetic Expression Evaluator	Version: 1.0
User's Manual	Date: 01/05/2026
EECS348-G10-UM	

4.4 Error Handling

The evaluator includes an error handling system that detects invalid syntax and runtime errors. This helps prevent incorrect calculations and improves user experience.

4.5 Tokenizer and Parser System

The software first converts the input expression into tokens and then parses the expression according to precedence rules before evaluation.

5. Troubleshooting

Problem: The program does not compile.

Possible Solution: Make sure a C++ compiler such as g++ is installed and available in the terminal.

Problem: The program displays "Syntax Error".

Possible Solution: Check the expression for missing operators, invalid symbols, or unmatched parentheses.

Problem: Division by zero error.

Possible Solution: Do not divide a number by zero.

Arithmetic Expression Evaluator	Version: 1.0
User's Manual	Date: 01/05/2026
EECS348-G10-UM	

Problem: The executable file does not run.

Possible Solution: Make sure the compile command is completed successfully before running the executable.

Problem: Incorrect expression results.

Possible Solution: Verify operator precedence and the placement of parentheses in the expression.

6. Examples

Example 1

Input: $2 + 3 * 4$

Output: 14

Example 2

Input: $(8 - 2) / 3$

Output: 2

Example 3

Input: $5 \% 2$

Arithmetic Expression Evaluator	Version: 1.0
User's Manual	Date: 01/05/2026
EECS348-G10-UM	
Output: 1	

<i>Example 4</i> Input: $2 ** 3$ Output: 8
--

<i>Example 5</i> Input: $-7 + 10$ Output: 3

<i>Example 6</i> Input: $(4 + 6) * (2 - 1)$ Output: 10
--

7. Glossary of terms

Expression: A mathematical statement containing numbers and operators.

Operator: A symbol used to perform a calculation, such as + or *.

Arithmetic Expression Evaluator	Version: 1.0
User's Manual	Date: 01/05/2026
EECS348-G10-UM	

Operand: A value used in an arithmetic operation.

Parser: A component that analyzes the structure of an expression.

Tokenizer: A component that breaks the input expression into smaller tokens.

Unary Operator: An operator that acts on a single value, such as -5.

Syntax Error: An error caused by an invalid expression format.

8. FAQ

Q: Does the evaluator support decimal numbers? A: Yes, floating-point expressions are supported.

Q: Can I use parentheses in expressions? A: Yes, parentheses are fully supported.

Q: What happens if I enter an invalid expression? A: The program displays an error message describing the issue.

Q: Does the evaluator follow PEMDAS rules? A: Yes, operator precedence is handled according to standard mathematical rules.

Q: Does the evaluator support negative numbers? A: Yes, unary plus and unary minus are supported.